

第七章：中间代码生成

常见的中间代码结构
语法制导方法概论

1. 生成中间代码的目的

- ◆ 1.便于优化
 - ❖ 让生成的目标代码效率更高
 - ❖ 优化不等同于对代码的精简和算法的优化

中间代码优化例1:

<pre>r=5; c=2*3.14*r;</pre>		<pre>r=5; c=31.4;</pre>
---------------------------------	---	-----------------------------

中间代码优化例2:

```
i=0;  
while(i<10)  
{ j=0;  
  while(j<10)  
  { A[i][j]=A[i][j]+1;  
    j++;  
  }  
  i++;  
}
```



1. 生成中间代码的目的

◆ 2. 便于移植

- ❖ 语言的移植是指语言在不同的机器上运行。
- ❖ 不同机器：指采用不同指令系统的机器，如Intel 80x86系列机器，Pentium机器，IBM大型机等



- ❖ 编译前端：(词法、语法、语义、中间代码生成) 与目标机无关的中间代码优化，移植时不需修改
- ❖ 编译后端：与目标机相关的中间代码优化、目标代码，移植时需要修改

2. 中间代码结构

- ◆ 2.1 逆波兰式
- ◆ 2.2 抽象语法树
- ◆ 2.3 三地址中间代码

2.1 逆波兰式（后缀表达式）

- 将运算对象写在前面，把运算符写在后面，因而也称后缀式。

程序设计语言中的表示	逆波兰表示
$a+b$	$ab+$
$a+b*c$	$abc*+$
$(a+b)*c$	$ab+c*$

- 逆波兰式的特点：

- ①逆波兰式中的变量的次序与中缀式中的变量的次序完全一致。
- ②逆波兰式中无括号。
- ③逆波兰式中的运算符已按计算顺序排列。

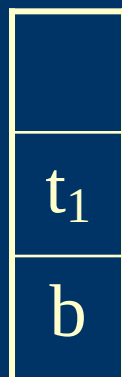
- ◆ 后缀式的计算机处理

- 后缀式的最大优点是易于计算机处理

- 处理过程：

从左到右扫描后缀式，每碰到运算对象就推进栈；碰到运算符就从栈顶弹出相应目数的运算对象施加运算，并把结果推进栈。最后的结果留在栈顶。

例：表达式 $b + c * d$ 的后缀式 $bcd*+$ 的求值过程



$$t_1 = c * d$$



$$t_2 = b + t_1$$

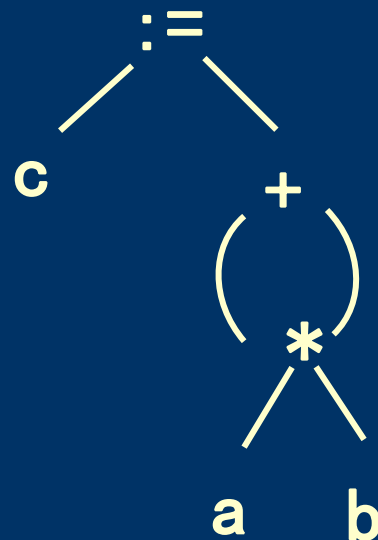
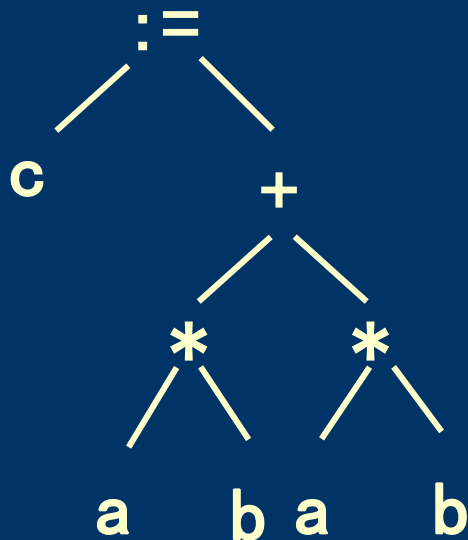
2.2 抽象语法树AGT和DAG

- 抽象语法树AGT (Abstract Grammar Tree) 可以显式地表示源程序的结构，是常用的一种标准中间代码形式。

例如： $c := a * b + a * b$ 的抽象语法树如下左图所示：

- 有向无环图DAG (Directed Acyclic Graph) 实际上是从抽象语法树发展而来，其主要优点是能够描述共享问题，如表达式的共享。

例如： $c := a * b + a * b$ 的无环有向图如下右图所示：



2.3 三地址中间代码

- ◆ 对于通用程序设计语言而言，程序的主体是表达式计算，所以针对表达式计算设计出三地址形式中间代码。
- ◆ 所谓三地址是指运算符的两个运算分量及该运算的结果的抽象地址。例如： $(+, x, y, t)$ 中， x, y, t 都是内存地址
- ◆ 其他结构的中间代码可以在表达式的基础上变形得到。
- ◆ 例如： $(\text{JMP}, -, -, L)$ 中，只有 L 一个地址；
 $(\text{ASSIG}, 100, -, x)$ ，有 $100, x$ 两个地址
- ◆ 常见的三地址中间代码有三元式和四元式两种。

- ◆ 例如：语句 $a := b * c + b / d$

三元式：三元式编号 (算符 op , ARG_1 , ARG_2)

(1) ($*$, b , c)

(2) ($/$, b , d)

(3) ($+$, (1), (2))

(4) ($:=$, (3), a)

三元式用编号表示运算结果

四元式: (算符 op , ARG_1 , ARG_2 , 运算结果 $RESULT$)

($*$, b , c , t_1)

($/$, b , d , t_2)

($+$, t_1 , t_2 , t_3)

($:=$, t_3 , $-$, a)

一般地，四元式用临时变量(临时存储单元)表示运算结果

◆ 三元式和四元式的比较:

- 三元式的优点是表示简单，但由于其运算结果反映在三元式的位置编号上，使得带中间代码移动或代码删除的优化工作变得复杂，所以三元式中间代码不利于优化；
- 四元式不依赖于位置，因此四元式结构的中间代码便于插入、删除和移动。

- ◆ 四元式的通用形式: (op, addr1, addr2, addr3)
- ◆ 其中op代表广义的运算符, 除算术运算、关系运算、逻辑运算外, 还可以表示赋值, 输入/输出, 函数调用, 跳转等;
ASSIG、READ、WRITE、RETURN、GE、ADD、JMP、CALL
- ◆ addr1表示第一操作数, 可以是普通变量、常数、标号、临时变量, 也可空;
(+, x, y, t) , (*, 100, z, t), (CALL, Lf, true, t), (*, t1, t2, t3), (JMP, -, -, L)
- ◆ addr2表示第二操作数, 可以是普通变量、常数、临时变量, 也可空;
(+, x, y, t) , (*, z, 100, t), (*, t1, t2, t3), (JMP, -, -, L)
- ◆ addr3表示结果地址, 可以是普通变量、标号、临时变量, 也可空;
(ASSIG, x, -, y), (JMP, -, -, L), (+, x, y, t), (WHILE, -, -, -)
- ◆ 当op是单目运算符时, 操作数优先放在addr1

◆ 本课程采用的四元式设计如下：

1. 算术、逻辑、关系运算符

ADDI、ADDF、SUBI、SUBF、MULTI、
MULTF、DIVI、DIVF、MOD、AND、
OR、EQ、NE、GT、GE、LT、LE

2. I/O操作

(READI, —, —, id)	整数输入
(READF , —, —, id)	实数输入
(WRITE, —, —, id)	输出id

3. 类型转换

(FLOAT, id_1 , —, id_2)

... $id_2 := \text{float} (id_1)$

4. 赋值

(ASSIG , id_1 , —, id_2)

... $id_2 := id_1$

5. 地址加

(AADD , id_1 , id_2 , id_3)

... $id_3 := \text{addr} (id_1) + id_2$

或 ([] , id_1 , id_2 , id_3)

6. 标号定义

(LABEL, —, —, label)

定义标号label

7. 条件/无条件转移

(JMP, —, —, label)

转向标号label

8. 过程/函数声明、调用

(CALL, f, true/false, Result)

过程或函数调用

(ENTRY, Label, Size, Level)

子程序入口

(RETURN,-,-,t/-)

返回

(ENDFUNC/ENDPROC,-,-,-)

子程序出口

9. 传送参数参数传递中间代码

(VARACT, t, Offset, Size)

(VALACT, t, Offset, Size)

(FUNCACT, t, Offset, 2)

(PROACT, t, Offset, 2)

10. 条件语句 (只考虑if语句)

(THEN, t, -, -)

(ELSE, -, -, -)

(ENDIF, -, -, -)

11. 循环语句 (只考虑while循环)

(WHILE, -, -, -)

(DO, t, -, -)

(ENDWHILE, -, -, -)

3. 语法制导方法

- ◆ 语法制导是在进行语法分析的同时完成相应的“语义”动作，这些语义动作对应一些子程序，这些子程序要完成和用户的需求相关联的任务。
- ◆ 具体实现方法是：在原文法的产生式的基础上，在产生式的适当位置增加一些语义符号。当编译器对某个串进行语法分析时，若遇到的是语法符号，就正常进行语法分析，若遇到的是语义符号，就执行相应的语义动作。当语法分析完成时，语义处理也相应完成，从而解决我们想要解决的问题。
- ◆ 语法制导技术在处理与规则相关联的任务中有着重要的应用

- ◆ **语法制导方法的种类**：语法制导方法依赖于具体的语法分析方法，因此可以分为自顶向下语法制导方法和自底向上语法制导方法。自顶向下语法制导方法中通常采用LL(1)分析方法为基础，而自底向上语法制导方法中通常以LR(1)分析方法为基础。
- ◆ **LL(1)语法制导方法**是在LL(1)文法的基础上加入语义动作符来表示相应的语义子程序，**语义动作符可以加在产生式右部的任何位置**。在进行LL(1)语法分析的过程中每遇到语义动作符，则调用相应的语义子程序来完成相应的语义处理。
- ◆ **LR(1)语法制导方法**是在LR(1)文法的基础上加入语义动作符来表示相应的语义子程序，每条产生式最多配有一个语义规则，即**语义动作符只能加在产生式的最右部**。在进行LR(1)语法分析的过程中，每当按照某个产生式的右部进行归约时，就调用该产生式右边的相应的语义子程序来完成相应的语义处理。

LL(1)语法制导方法的实例

◆ 设有如下LL(1)文法:

G:

$S \rightarrow A B$

$A \rightarrow a A \mid b$

$B \rightarrow b B \mid c$

要求: 应用语法制导翻译方法完成如下任务:

对输入文法 G 的任意句子 L, 输出 L 中b 串的长度。

如串 abbbc 是 G 的句子, 则该处理器将输出3。

G:

$S \rightarrow AB$

$A \rightarrow aA$

$| b$

$B \rightarrow bB$

$| c$

G的原始LL(1)分析表

	a	b	c	#
S	$S \rightarrow AB$	$S \rightarrow AB$		
A	$A \rightarrow aA$	$A \rightarrow b$		
B		$B \rightarrow bB$	$B \rightarrow c$	

◆ G 的LL(1)语法制导文法如下:

G: $S \rightarrow \text{\textit{\#Init\#}} AB$

$A \rightarrow aA \mid b \text{\textit{\#Add\#}}$

$B \rightarrow b \text{\textit{\#Add\#}} B \mid c \text{\textit{\#Out_Val\#}}$

其中, 语义符号Init、Add、Out-Val加了#与文法的语法符号相区分。

各动作符对应的语义子程序如下:

Init() { m = 0;}

Out_Val() { printf ("%d" , m);}

Add() { m++ ;}

G[S]:

$S \rightarrow \text{\textcolor{brown}{\#nit\#}} A B$

$A \rightarrow a A \mid b \text{\textcolor{brown}{\#Add\#}}$

$B \rightarrow b \text{\textcolor{brown}{\#Add\#}} B \mid c \text{\textcolor{brown}{\#Out_Val\#}}$

G的带动作符的LL(1)分析表

	a	b	c	#
S	$S \rightarrow \text{\textcolor{brown}{\#nit\#}} A B$	$S \rightarrow \text{\textcolor{brown}{\#nit\#}} A B$		
A	$A \rightarrow a A$	$A \rightarrow b \text{\textcolor{brown}{\#Add\#}}$		
B		$B \rightarrow b \text{\textcolor{brown}{\#Add\#}} B$	$B \rightarrow c \text{\textcolor{brown}{\#Out_Val\#}}$	

■语法制导的实现过程（1）

例 $G[s]$:

[1] $S \rightarrow$	#Init#	$A B$	$\{a\}$	[2] $A \rightarrow$	$a A$	$\{a\}$
[3] $A \rightarrow$	b	#Add#	$\{b\}$	[4] $B \rightarrow$	b	#Add#
[5] $B \rightarrow$	c	#Out_Val#	$\{c\}$		B	$\{b\}$

对给定的终极字符串**abbbc**，分析过程：

S#	abbbc #	Derivation
#init# $A B$ #	abbbc #	$\$ \#Init \#$
$A B$ #	abbbc #	Derivation
$a A B$ #	abbbc #	Match
$A B$ #	bbbc #	Derivation
b #Add# B #	bbbc #	Match
#Add# B #	bbc #	#Add#
B #	bbc #	

■语法制导的实现过程（2）

例 $G[s]$:

[1] $S \rightarrow$	#Init#	$A B$	{a}	[2] $A \rightarrow$	$a A$	{a}
[3] $A \rightarrow$	b	#Add#	{b}	[4] $B \rightarrow$	b	#Add# B {b}
[5] $B \rightarrow$	c	#Out_Val#	{c}			

对给定的终极字符串abbbc, 分析过程(续) :

B #	bbc #	Derivation
b #Add# B #	bbc #	Match
#Add# B #	bc #	#Add#
B #	bc #	Derivation
b#Add# B #	bc #	Match
#Add#B #	c #	#Add#
B #	c #	Derivation
c #Out_Val# #	c #	Match
#Out_Val# #	#	#Out_Val#
#	#	Success

3. 中间代码生成中的几个问题

◆ (一) 三地址代码中地址的表示

◆ “地址” 有三类：标号类、数值类、地址类

- ◆ 标号类的语义信息是相应的标号值，包括过程/函数体的入口标号；
(JMP, -, -, L), (CALL, Lf, true, t)
- ◆ 数值类的语义信息就是该数据值； (+, 100, x, t)
- ◆ 地址类的语义信息: 例如 (+, 100, x, t1), ([], A, t2, t3)中的x, t1, A, t2, t3等。
 - ◆ 如果符号表一直保留到目标代码生成阶段，地址可用该符号在符号表中的地址；
 - ◆ 如果符号表不保留到目标代码生成阶段，则用层数、偏移和访问方式(分为直接访问方式和间接访问方式)来表示地址。变参变量（指针变量）以及代表复杂变量地址的临时变量属于间接访问方式。
 - ◆ 临时变量不在符号表中，其地址中的层数取-1，偏移用编号代替，存取方式取决于产生临时变量的运算符：如上例中t1是dir，t3是indir。

3. 中间代码生成中的几个问题

◆ (二) 语义信息的提取与保存

- ◆ 在生成中间代码时需要用到标识符的类型、地址等信息，以便完成类型检查和代码生成的工作。
- ◆ 这些信息可在符号表中获得。
- ◆ 如果符号表保留到目标代码生成阶段，可用符号在符号表中的地址代替，需要时通过查表获得相应的语义信息；否则，需要把对应符号的语义信息放入四元式中；其中类型信息在生成完中间代码后不再需要，所以不必把类型信息记入四元式中，只记地址即可。

如：3.5+i 对应的四元式为：

(ADDF, 3.5, (i.level,i.off,dir), (-1,t3,dir))

3. 中间代码生成中的几个问题

- ◆ (三) 语义栈Sem的结构及其操作
- ◆ Sem的结构 (类型、地址) 二元组

intptr	(x.level, x.off, dir)

语义栈Sem

- ◆ Sem的操作
 - push(id) : 将id的类型及地址信息压入Sem
 - pop(n) : 从栈顶依次弹出n的元素

3. 中间代码生成中的几个问题

◆ (四) 语法制导中用到的一些核心语义符号

GenCode(ω)

AddNext

Assig

ThenIf

ElseIf

EndIf

StartWhile

DoWhile

EndWhile

Entry

Retrun

EndPro/EndFunc

CallHead

CallTail

EvlParam